



LeetCode 14: Longest Common Prefix

1. Problem Title & Link

Title: LeetCode 14: Longest Common Prefix

Link: <https://leetcode.com/problems/longest-common-prefix/>

2. Problem Statement (Short Summary)

Given an array of strings `strs`, find the **longest common prefix** shared by all strings.

A prefix means:

flower

^ ^ ^ ^

characters starting from index 0.

If no common prefix exists, return:

""

Example

Input:

`strs = ["flower", "flow", "flight"]`

Output:

`"fl"`

Because:

flower

flow

flight

Common starting characters:

f

l

After that:

`o ≠ i`

So answer is:

`"fl"`

3. Examples (Input → Output)

Example 1

Input:

`["flower", "flow", "flight"]`

Output:

`"fl"`

**Example 2**

Input:

["dog", "racecar", "car"]

Output:

""

Explanation:

$d \neq r \neq c$

No common prefix.

Example 3

Input:

["interview", "internet", "internal"]

Output:

"inter"

Example 4

Input:

["a"]

Output:

"a"

4. Constraints

$1 \leq \text{strs.length} \leq 200$

$0 \leq \text{strs}[i].\text{length} \leq 200$

`strs[i]` contains only lowercase English letters

Important:

Need longest common PREFIX

Not substring.

5. Core Concept (Pattern / Topic)**String Traversal**

Secondary Topics:

- Vertical Scanning



- Character Comparison

6. Thought Process (Step-by-Step Explanation)

Understanding Prefix

For:

flower

Prefixes are:

f

fl

flo

flow

flowe

flower

For:

flow

Prefixes are:

f

fl

flo

flow

Common prefixes:

f

fl

flo

flow

For:

flight

Prefixes are:

f

fl

fli

flig

...

Common among ALL strings:

f



fl

Answer:

fl

Key Observation

Compare characters column by column.

Instead of comparing whole strings:

flower

flow

flight

Compare:

Index 0

f

f

f

Match.

Index 1

l

l

l

Match.

Index 2

o

o

i

Mismatch.

Stop.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
flower
flow
flight
```



Compare vertically:

f f f ✓

l l l ✓

o o i ✗

Stop.

Answer:

fl

Another Example

dog

racecar

car

d

r

c

Mismatch immediately.

Answer:

""

8. Pseudocode (Language Independent)

```
prefix = first string

for every other string
    while string does not start with prefix
        remove last character from prefix
    if prefix becomes empty
        return ""

return prefix
```

9. Code Implementation

Python

```
class Solution:
    def longestCommonPrefix(self, strs):

        prefix = strs[0]

        for s in strs[1:]:

            while not s.startswith(prefix):
```



```
        prefix = prefix[:-1]

        if not prefix:
            return ""

    return prefix
```

Java

```
class Solution {

    public String longestCommonPrefix(String[] strs) {

        String prefix = strs[0];

        for(int i = 1; i < strs.length; i++) {

            while(!strs[i].startsWith(prefix)) {

                prefix =
                    prefix.substring(
                        0,
                        prefix.length() - 1
                    );

                if(prefix.isEmpty())
                    return "";

            }

        }

        return prefix;

    }

}
```

10. Time & Space Complexity

Time Complexity

$O(n * m)$

where:

n = number of strings

m = length of shortest string

Space Complexity

$O(1)$



No extra data structures.

11. Common Mistakes / Edge Cases

Mistake 1

Finding common substring instead of prefix.

Example:

abcxyz

xyzabc

Common substring:

abc

But prefix:

""

Answer should be empty.

Mistake 2

Comparing beyond shortest string length.

Example:

["ab", "a"]

Need answer:

"a"

Mistake 3

Not handling single string input.

Example:

["hello"]

Answer:

"hello"

12. Detailed Dry Run

Input:

["flower", "flow", "flight"]

Initial:

prefix = "flower"

Compare with:

flow



Remove characters:

flower

flowe

flow

Now:

flow

matches.

Compare with:

flight

Current prefix:

flow

Reduce:

flow

flo

fl

Now:

flight starts with "fl"

Answer:

fl

13. Common Use Cases (Real-Life / Interview)

Search Engines

Finding common query prefixes.

Auto Completion

Example:

internet

internal

interval

Common prefix:

inter

File Systems

Finding common directory paths.

DNA Analysis



Finding common sequence starts.

14. Common Traps (Important!)

Trap 1

Confusing prefix with substring.

Trap 2

Using nested comparisons unnecessarily.

Trap 3

Forgetting empty prefix case.

Example:

dog

racecar

car

Answer:

""

15. Builds To (Related LeetCode Problems)

LC 14 - Longest Common Prefix



LC 28 - Find Index of First Occurrence



LC 58 - Length of Last Word



LC 211 - Design Add and Search Words



LC 208 - Implement Trie

These develop string-processing skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Horizontal Scanning	$O(n \times m)$	$O(1)$	Best
Vertical Scanning	$O(n \times m)$	$O(1)$	Best
Sorting	$O(n \log n)$	$O(1)$	Good



Trie	$O(\text{total chars})$	$O(\text{total chars})$	Advanced
------	-------------------------	-------------------------	----------

Approach 1: Horizontal Scanning (Chosen)

Start with first string as prefix.

Keep shrinking.

Most common interview solution.

Approach 2: Vertical Scanning

Compare column by column.

Very intuitive.

Approach 3: Sorting

Sort strings.

Only compare:

first string

last string

Common trick.

17. Why This Solution Works (Short Intuition)

Any common prefix must be a prefix of the first string.

Start with the entire first string and continuously shrink it until every string starts with it.

The remaining string is the longest common prefix.

18. Variations / Follow-Up Questions

Follow-Up 1

Find longest common suffix.

Example:

walking

talking

barking

Answer:

king

Follow-Up 2

Find longest common substring.



Different problem.

Much harder.

Follow-Up 3

Implement using Trie.

Common interview follow-up.

Follow-Up 4

Find common prefix among millions of strings.

Discuss Trie optimization.

Interview Takeaway

The key pattern is:

Shrink Candidate Until Valid

Start with a possible answer:

first string

and keep reducing it until all strings satisfy the condition.

Whenever you hear:

Common Prefix

Shared Beginning

Autocomplete

think:

String Traversal

Prefix Matching

Trie (Advanced)